SUPPLEMENTARY MATERIAL

Supplementary Material for "Stochastic Volatility Models with Long Memory: A Comprehensive Review of Theory, Inference and Applications"

Qiang Zheng[1] | **Shelton Peiris**[2] | **Jennifer S.K. Chan**[3] | **Zhenglyu Huang**[4]

Appendix A: GARMA package for Gegenbaur return SV model

| A1. Model fitting

The program for running a Gegenbaur return SV model to the built-in AirPassengers data using the GARMA package is provided below:

```
> install.packages("garma");  install.packages("forecast");  install.packages("tseries")
> library(garma); library(forecast); library(tseries)

# Example 1: AirPassengers (built-in)
# Load data Example 1: AirPassengers (built-in)
> data("AirPassengers")
> y <- AirPassengers   # stabilize variance and remove trend

# Remove deterministic seasonality
> fit_season <- tslm(y ~ trend + season)
> y_detrend <- residuals(fit_season)

# Plot data, ACF and Periodogram
> plot(y_detrend, main = "detrend(AirPassengers)")
> acf(y_detrend, lag.max=100, main = "ACF: detrend(diff log AirPassengers)")
> spec.pgram(as.numeric(y_detrend), main = "Periodogram: detrend(AirPassengers)")

# Fit GARMA (1 Gegenbauer factor)
> fit_garma <- garma(y_detrend, order = c(0,0,0), k = 1)
> summary(fit_garma)

Call:
garma(x = y_detrend, order = c(0, 0, 0), k = 1)

Mean term was fitted.
No Drift (trend) term was fitted.

Summary of Gegenbauer Time Series model.
```

```
Fit using Whittle method, with Order=(0,0,0) k=1
Optimisation Method(s):  cobyla, solnp

Coefficients:
      intercept        u1      fd1
      2.368e-15  0.887003  0.50000
s.e.  2.100e+00  0.007977  0.03287

                         Factor1
Gegenbauer frequency:     0.0764
Gegenbauer Period:       13.0904
Gegenbauer Exponent:      0.5000

sigma^2 estimated as 38.7535: log likelihood = -1140.446963, aic = 2286.893926

> # Compare with ARIMA
> auto.arima(y_detrend)
Series: y_detrend
ARIMA(2,1,1)(0,1,0)[12]

Coefficients:
         ar1     ar2      ma1
      0.5960  0.2143  -0.9819
s.e.  0.0888  0.0880   0.0292

sigma^2 = 132.3:  log likelihood = -504.92
AIC=1017.85   AICc=1018.17   BIC=1029.35
```

| A2. Data simulation for one-factor GARMA model

The R codes for simulating a time series following GARMA model using Eqs. (**??**) to (**??**) are provided below:

```
gegenbauer_sim <- function(n = 1500, u = 0.7, d = 0.35, sigma = 1, M = 50) {

# White noise;  M: truncation lag for filter
e <- rnorm(n + M, sd = sigma)

  # Compute Gegenbauer coefficients recursively
  psi <- numeric(M)
  psi[1] <- 1
  psi[2] <- 2*u*d

  for (k in 3:M) {
    psi[k] <- (2*u*(k-2+d)*psi[k-1] - (k-3+2*d)*psi[k-2]) / (k-1)
  }

  # Apply filter (convolution)
  y <- filter(e, filter = psi, method = "convolution", sides = 1)

  # Remove burn-in
  y <- y[(M+1):(M+n)]
  return(as.numeric(y))
}

set.seed(123)
y <- gegenbauer_sim_spec(n = 1500, u = 0.7, d = 0.35)
```

| A3. Data simulation for two-factor GARMA model

The R codes for simulating a time series following two-factor GARMA model are provided below:

```
gegenbauer_sim_k2 <- function(n=1500, u = c(0.7,0.3), d = c(0.35,0.25), sigma = 1, M = 100) {

# Compute Gegenbauer coefficients for one factor; M: truncating lag for filter
compute_psi <- function(u, d, M) {
  psi <- numeric(M)
  psi[1] <- 1
  psi[2] <- 2 * u * d

for (k in 3:M) {
  psi[k] <- (2 * u * (k - 2 + d) * psi[k - 1] - (k - 3 + 2 * d) * psi[k - 2]) / (k - 1)
  }
return(psi)
  }

# Compute psi for both factors
psi1 <- compute_psi(u[1], d[1], M)
psi2 <- compute_psi(u[2], d[2], M)

# Convolution of filters
psi_total <- convolve(psi1, rev(psi2), type = "open")

# Truncate to M (important for stability)
psi_total <- psi_total[1:M]

# White noise
e <- rnorm(n + M, sd = sigma)

# Apply filter
y <- filter(e, filter = psi_total, method = "convolution", sides = 1)

# Remove burn-in
y <- y[(M + 1):(M + n)]
return(as.numeric(y))
}

set.seed(123)
y <- gegenbauer_sim_k2(n = 2000, u = c(0.2, 0.7), d = c(0.4, 0.3), M = 200)
```

Appendix B. RStan programs for Gegenbaur SV model

| B1. Data simulation for GSV model

```
library(rstan)
set.seed(100)

# Simulate a time series y_t from Gegenbauer SV model
# N: sample size
# mu: mean of h
# sigma: standard error of eta_t
# u,j: coefficients in Gegenbauer polynomial
# J: Truncation value when approximating infinite long memory
N = 2000
mu = log(0.0001); sigma = 2
```

```
u = 0.8; d = 0.3; J = 50

y = numeric(N)
h = numeric(N)
lambda = numeric(J)

eta = rnorm(N+J,0,sigma)

lambda[1] = 2*u*d
lambda[2] = 2*u*((d-1)/2+1)*lambda[1] - (2*(d-1)/2+1)
for (j in 3:J){
  lambda[j] = 2*u*((d-1)/j+1)*lambda[j-1] - (2*(d-1)/j+1)*lambda[j-2];
}

for (t in 1:N){
  h[t] = sum(eta[(t+J):t]*c(1,lambda)) + mu
  y[t] = rnorm(1,0,exp(h[t]/2))
}
acf(h); pacf(h)

#Create model specification in Stan
code = '
data {
  int<lower=0> N;
  vector[N] y;
  int<lower=3> J;
}

parameters {
  real<lower=-1,upper=1> u;
  real<lower=0, upper=0.5> d;
  real mu;
  real<lower=0> sigma;
  real eta[N];
}

transformed parameters{
  real lambda[J];
  real h[N-J];
  real<lower=0, upper=1> a;
  real<lower=0, upper=1> b;
  lambda[1] = 2*u*d;
  lambda[2] = 2*u*((d-1)/2+1)*lambda[1] - (2*(d-1)/2+1);
  a = (1+u)/2;
  b = 2*d;
  for (j in 3:J){
    lambda[j] = 2*u*((d-1)/j+1)*lambda[j-1] - (2*(d-1)/j+1)*lambda[j-2];
  }
  for (i in 1:(N-J)){
    h[i] = eta[i+J]+mu;
    for (j in 1:J)
      h[i] += eta[i+J-j]*lambda[j];
  }
}

model {
  a ~ beta(2,2);
  b ~ beta(2,2);
  mu ~ normal(-5,2);
  sigma ~ gamma(5,10);
  eta ~ normal(0, sigma);
```

```
  for (i in 1:(N-J)){
    y[i+J] ~ normal(0, exp(h[i]/2));
  }
}'

#compile the model
mod = stan_model(model_code = code)

test_data = list(
  N = length(y),
  y = y,
  J = 30
)

#MCMC sampling
fit = sampling(mod, data = test_data, iter = 10000, warmup = 5000,chains = 1)
#compute posterior means of u, d and sigma
mean(extract(fit,'u')$u)
mean(extract(fit,'d')$d)
mean(extract(fit,'sigma')$sigma)
```

| B2. Model fitting for standard and two-factor GSV models

```
library(rstan)
library(quantmod)
library(latex2exp)
set.seed(100)

#function to compute Cauchy product of series
cauchy_prod <- function(a) {
  if (dim(a)[2] == 2) {
    lambda = numeric(dim(a)[1])
    for (k in 1:(dim(a)[1])) {
      lambda[k] = sum(a[1:k, 1] * a[k:1, 2])
    }
    return(lambda)
  }
  if (dim(a)[2] > 2) {
    return(cauchy_prod(cbind(cauchy_prod(a[, -1]), a[, 1])))
  }
  else{
    stop('Only one series is provided')
  }
}

#function to calculate coefficients of analytical expantion of
#\prod (1-2u_iB + B^2)^-d_i up tp order J
g_coef <- function(u, d, J) {
  if (length(u) != length(d)) {
    stop('The length of u(',
         length(u),
         ') does not match length of d(',
         length(d),
         ').')
  }
  lambda = matrix(0, nrow = J, ncol = length(d))
  lambda[1, ] = 1
  lambda[2, ] = 2 * u * d
```

```
  for (j in 3:J) {
    lambda[j, ] = 2 * u * ((d - 1) / (j-1) + 1) * lambda[j - 1, ] -
                  (2 * (d - 1) /  (j-1) + 1 ) * lambda[j - 2, ]

  }
  if (length(u) == 1) {
    return(as.vector(lambda))
  }
  else{
    return(cauchy_prod(lambda))
  }
}


#Create model specification in Stan
code = code = r'(
functions{
  vector cauchy_prod(matrix a) {
    if (dims(a)[2] == 2) {
        vector[dims(a)[1]] lambda;
        lambda = to_vector(rep_array(0, dims(a)[1]));
      for (k in 1:(dims(a)[1])) {
        for (i in 1:k){
          lambda[k] += a[i, 1] * a[k-i+1, 2];
        }
      }
      return lambda;
    } else {
      return cauchy_prod(append_col(cauchy_prod(a[, 2:dims(a)[2]]), a[, 1]));
    }
  }

  vector g_coef(row_vector u, row_vector d, int J) {
    matrix[J + 1 ,size(u)] lambda;
    lambda[1, ] = to_row_vector(rep_array(1, size(u)));
    lambda[2, ] = 2 .* u .* d;
    for (j in 3:(J+1)) {
      lambda[j, ] = 2 * u .* ((d - 1) / (j-1) + 1) .* lambda[j - 1, ] -
                    (2 * (d - 1) /  (j-1) + 1 ) .* lambda[j - 2, ];
    }
    if (size(u) == 1) {
      return to_vector(lambda);
    }
    else{
      return cauchy_prod(lambda);
    }
  }
}

data {
  int<lower=0> N;
  int<lower=1> k;
  vector[N] y;
  int<lower=3> J;
}

parameters {
  ordered[k] lu;
  row_vector<lower=0, upper=0.5>[k] d;
  real mu;
  real<lower=0> sigma;
```

```
  real eta[N];
}

transformed parameters{
  vector[J+1] lambda;
  real h[N-J];
  row_vector<lower=0, upper=1>[k] a;
  row_vector<lower=-1, upper=1>[k] u;
  u = 2/(1+exp(-lu'))-1;
  lambda = g_coef(u ,d , J);
  a = 2*d;
  for (i in 1:(N-J)){
    h[i] = mu;
    for (j in 0:J){
      h[i] += eta[i+J-j]*lambda[j+1];
    }
  }
}

model {
  a ~ beta(2,2);
  lu ~ normal(0,5);
  mu ~ normal(-5,2);
  sigma ~ gamma(5,10);
  eta ~ normal(0, sigma);
  for (i in 1:(N-J)){
    y[i+J] ~ normal(0, exp(h[i]/2));
  }
}

)'

#compile the model
mod = stan_model(model_code = code)

################One-Factor GSV
# Simulate a time series y_t from Gegenbauer SV model
# N: sample size
# mu: mean of h
# sigma: standard error of eta_t
# u,d: coefficients in Gegenbauer polynomial
# J: Truncation value when approximating infinite long memory
N = 2000
mu = log(0.0001)
sigma = 2
u = 0.8
d = 0.3
J = 50

y = numeric(N)
h = numeric(N)
lambda = numeric(J)

eta = rnorm(N+J,0,sigma)

lambda = g_coef(u, d, J+1)

for (t in 1:N){
  h[t] = sum(eta[(t+J):t]*lambda) + mu
  y[t] = rnorm(1,0,exp(h[t]/2))
}
```

```
#pdf(file = 'plot/GSV.pdf')
plot(h, type = 'l', cex.main = 2, cex.lab = 1.5)
acf(h, main = 'ACF: h', cex.main = 2, cex.lab = 1.5)
spec.pgram(h, main ='Periodogram: h', cex.main = 2, cex.lab = 1.5)
plot(y, type = 'l', cex.main = 1.5, cex.lab = 1.5)
acf(y, main = 'ACF: y', cex.main = 1.5, cex.lab = 1.5)
spec.pgram(y, main = 'Periodogram: y', cex.main = 1.5, cex.lab = 1.5)
plot(log(y^2), type = 'l', cex.main = 1.5, cex.lab = 1.5)
acf(log(y^2), main = TeX(r'(ACF: $\log(y^2)$)'), cex.main = 1.5, cex.lab = 1.5)
spec.pgram(log(y^2), main = TeX(r'(Periodogram: $\log(y^2)$)'), cex.main = 1.5, cex.lab = 1.5)
#dev.off()

test_data = list(
  N = length(y),
  k = 1,
  y = y,
  J = 30
)

#MCMC sampling
fit = sampling(mod, data = test_data, iter = 10000, warmup = 5000,chains = 1)
#compute posterior means of u, d and sigma
mean(extract(fit,'u')$u)
mean(extract(fit,'d')$d)
mean(extract(fit,'sigma')$sigma)
sd(extract(fit,'u')$u)
sd(extract(fit,'d')$d)
sd(extract(fit,'sigma')$sigma)

################Two-Factor GSV
# Simulate a time series y_t from Gegenbauer SV model
# N: sample size
# mu: mean of h
# sigma: standard error of eta_t
# u_i,d_i: coefficients in Gegenbauer polynomial
# J: Truncation value when approximating infinite long memory
N = 2000
mu = log(0.0001)
sigma = 2
u = c(0.8, -0.5)
d = c(0.3,0.4)
J = 50

y = numeric(N)
h = numeric(N)
lambda = numeric(J)

eta = rnorm(N+J,0,sigma)

lambda = g_coef(u, d, J+1)

for (t in 1:N){
  h[t] = sum(eta[(t+J):t]*lambda) + mu
  y[t] = rnorm(1,0,exp(h[t]/2))
}

#pdf(file = 'plot/GSV2.pdf')
plot(h, type = 'l', cex.main = 2, cex.lab = 1.5)
acf(h, main = 'ACF: h', cex.main = 2, cex.lab = 1.5)
spec.pgram(h, main ='Periodogram: h', cex.main = 2, cex.lab = 1.5)
plot(y, type = 'l', cex.main = 1.5, cex.lab = 1.5)
```

```
acf(y, main = 'ACF: y', cex.main = 1.5, cex.lab = 1.5)
spec.pgram(y, main = 'Periodogram: y', cex.main = 1.5, cex.lab = 1.5)
plot(log(y^2), type = 'l', cex.main = 1.5, cex.lab = 1.5)
acf(log(y^2), main = TeX(r'(ACF: $\log(y^2)$)'), cex.main = 1.5, cex.lab = 1.5)
spec.pgram(log(y^2), main = TeX(r'(Periodogram: $\log(y^2)$)'), cex.main = 1.5, cex.lab = 1.5)
#dev.off()

test_data = list(
  N = length(y),
  k = 2,
  y = y,
  J = 30
)

fit2 = sampling(mod, data = test_data, iter = 10000, warmup = 5000,chains = 1)
apply(extract(fit2,'u')$u,2,mean)
apply(extract(fit2,'d')$d,2,mean)
mean(extract(fit2,'sigma')$sigma)
apply(extract(fit2,'u')$u,2,sd)
apply(extract(fit2,'d')$d,2,sd)
sd(extract(fit2,'sigma')$sigma)
```

Appendix C. R code for HAR model

For reproducibility and implementation, it is convenient to work with log-volatility proxies in R. The following template provides a minimal example for computing RV_t and fitting a simple HAR benchmark; the same data structures can be used to implement ARFIMA-SV and GSV estimators.

```
# Load required packages
library(quantmod)
library(xts)

# Download S&P 500 data
getSymbols("^GSPC", src = "yahoo", from = "2000-01-01", to = "2025-12-31")
spx <- GSPC

# Daily log returns
ret <- dailyReturn(Cl(spx), type = "log")
ret <- na.omit(ret)
ret <- ret[ret!=0]

# Realized volatility proxy and log transform
RV    <- ret^2
logRV <- log(RV)
logRV <- na.omit(logRV)

# Construct HAR regressors and estimate a simple HAR model
y    <- as.numeric(logRV)
n    <- length(y)
y_w  <- filter(y, rep(1/5, 5),  sides = 1)
y_m  <- filter(y, rep(1/22,22), sides = 1)

har_df <- data.frame(
  y      = y,
  y_lag1 = c(NA, y[-n]),
  y_w    = c(NA,as.numeric(y_w)[-n]),
  y_m    = c(NA,as.numeric(y_m)[-n])
```

```
)
har_df <- na.omit(har_df)
har_fit <- lm(y ~ y_lag1 + y_w + y_m, data = har_df)
summary(har_fit)
MSE = mean(har_fit$residuals^2); MSE
```

The GSV models can be implemented by combining this data structure with Whittle-based estimation of Gegenbauer parameters, following the references cited below.

```
library(rstan)
library(quantmod)
library(latex2exp)
set.seed(100)

#Create model specification in Stan
code = r'(
functions{
  vector cauchy_prod(matrix a) {
    if (dims(a)[2] == 2) {
        vector[dims(a)[1]] lambda;
        lambda = to_vector(rep_array(0, dims(a)[1]));
      for (k in 1:(dims(a)[1])) {
        for (i in 1:k){
          lambda[k] += a[i, 1] * a[k-i+1, 2];
        }
      }
      return lambda;
    } else {
      return cauchy_prod(append_col(cauchy_prod(a[, 2:dims(a)[2]]), a[, 1]));
    }
  }

  vector g_coef(row_vector u, row_vector d, int J) {
    matrix[J + 1 ,size(u)] lambda;
    lambda[1, ] = to_row_vector(rep_array(1, size(u)));
    lambda[2, ] = 2 .* u .* d;
    for (j in 3:(J+1)) {
      lambda[j, ] = 2 * u .* ((d - 1) / (j-1) + 1) .* lambda[j - 1, ] -
                    (2 * (d - 1) /  (j-1) + 1 ) .* lambda[j - 2, ];
    }
    if (size(u) == 1) {
      return to_vector(lambda);
    }
    else{
      return cauchy_prod(lambda);
    }
  }
}

data {
  int<lower=0> N;
  int<lower=1> k;
  vector[N] y;
  int<lower=3> J;
}

parameters {
  ordered[k] lu;
  row_vector<lower=0, upper=0.5>[k] d;
  real mu;
  real<lower=0> sigma;
```

```
  real eta[N];
}

transformed parameters{
  vector[J+1] lambda;
  real h[N-J];
  row_vector<lower=0, upper=1>[k] a;
  row_vector<lower=-1, upper=1>[k] u;
  u = 2/(1+exp(-lu'))-1;
  lambda = g_coef(u ,d , J);
  a = 2*d;
  for (i in 1:(N-J)){
    h[i] = mu;
    for (j in 0:J){
      h[i] += eta[i+J-j]*lambda[j+1];
    }
  }
}

model {
  a ~ beta(2,2);
  lu ~ normal(0,5);
  mu ~ normal(-5,2);
  sigma ~ gamma(5,10);
  eta ~ normal(0, sigma);
  for (i in 1:(N-J)){
    y[i+J] ~ normal(0, exp(h[i]/2));
  }
}

)'

mod = stan_model(model_code = code)

code = '
data {
  int<lower=0> N;
  vector[N] y;
  int<lower=3> J;
}

parameters {
  real<lower=0, upper=0.5> d;
  real mu;
  real<lower=0> sigma;
  real eta[N];
}

transformed parameters{
  real h[N-J];
  vector[J+1] lambda;
  lambda[1] = 1;
  real<lower=0, upper=1> b;
  b = 2*d;
  for (j in 1:J){
      lambda[j+1] = lambda[j]*(-d-j+1)/j;
  }
  for (i in 1:(N-J)){
    h[i] = mu + eta[i+J];
    for (j in 1:J){
      lambda[j+1] = -lambda[j]*(-d-j+1)/j;
```

```
      h[i] += eta[i+J-j]*lambda[j+1];
    }
  }
}

model {
  b ~ beta(2,2);
  mu ~ normal(-5,2);
  sigma ~ gamma(5,10);
  eta ~ normal(0, sigma);
  for (i in 1:(N-J)){
    y[i+J] ~ normal(0, exp(h[i]/2));
  }
}'

mod2 = stan_model(model_code = code)

#Case study
getSymbols("^GSPC", src = "yahoo", from = "2000-01-01", to = "2025-12-31")

# Daily log returns
ret <- dailyReturn(Cl(GSPC), type = "log")
ret <- na.omit(ret)
ret <- ret[ret!=0]
t = index(ret)

pdf(file = 'plot/sp500.pdf')
plot(t, ret, main = "Daily return of S&P 500", xlab = 'Date',
                    ylab = 'r', type = 'l', cex.main = 2, cex.lab = 1.5)
acf(ret, lag.max=100, main = "ACF: S&P 500", cex.main = 2, cex.lab = 1.5)
spec.pgram(ret, main = "Periodogram: S&P 500", cex.main = 2, cex.lab = 1.5)
plot(t, ret^2, main = "Squared daily return of S&P 500", xlab = 'Date',
                       ylab = TeX(r'($r^2$)'), type = 'l', cex.main = 2, cex.lab = 1.5)
acf(ret^2, lag.max=100, main =  TeX(r'(ACF: $($S&P 500$)^2$)'), cex.main = 2, cex.lab = 1.5)
spec.pgram(ret^2, main = TeX(r'(Periodogram: $($S&P 500$)^2$)'), cex.main = 2, cex.lab = 1.5)
plot(t, log(ret^2), main = "Log of squared daily return of S&P 500", xlab = 'Date',
                           ylab = TeX(r'($y$)'), type = 'l', cex.main = 2, cex.lab = 1.5)
acf(log(ret^2), lag.max=100, main = "ACF: y", cex.main = 2, cex.lab = 1.5)
spec.pgram(log(ret^2), main = "Periodogram: y", cex.main = 2, cex.lab = 1.5)
dev.off()

# Realized volatility proxy and log transform
RV    <- ret^2
logRV <- log(RV)
logRV <- na.omit(logRV)

# Construct HAR regressors and estimate a simple HAR model
y    <- as.numeric(logRV)
n    <- length(y)
y_w  <- filter(y, rep(1/5, 5),  sides = 1)
y_m  <- filter(y, rep(1/22,22), sides = 1)

har_df <- data.frame(
  y      = y,
  y_lag1 = c(NA, y[-n]),
  y_w    = c(NA,as.numeric(y_w)[-n]),
  y_m    = c(NA,as.numeric(y_m)[-n])
)
har_df <- na.omit(har_df)
har_fit <- lm(y ~ y_lag1 + y_w + y_m, data = har_df)
summary(har_fit)
```

```
MSE = mean(har_fit$residuals^2)

# Define truncation value of long memory
J = 30
r = as.numeric(ret)

test_data = list(
  N = length(r),
  k = 1,
  y = r,
  J = J
)

fit1 = sampling(mod, data = test_data, iter = 10000, warmup = 5000,chains = 1)
h1 = apply(extract(fit1,'h')$h,2,mean)
MSE1 = mean((h1 - 1.27 - log(ret[-(1:J)]^2))^2)

test_data = list(
  N = length(r),
  k = 2,
  y = r,
  J = J
)

fit2 = sampling(mod, data = test_data, iter = 10000, warmup = 5000,chains = 1)
h2 = apply(extract(fit2,'h')$h,2,mean)
MSE2 = mean((h2 - 1.27 - log(ret[-(1:J)]^2))^2)



fit3 = sampling(mod2, data = test_data, iter = 10000, warmup = 5000,chains = 1)
h3 = apply(extract(fit3,'h')$h,2,mean)
MSE3 = mean((h3 - 1.27 - log(ret[-(1:J)]^2))^2)
pdf(file = 'plot/sp500_fitted.pdf',height=5, width=8)
layout(matrix(1:2, nrow=2, byrow=TRUE),
       heights = c(0.95, 0.05))
par(mar=c(4,2.5,1,1))
plot(t,y,type = 'l',col = 'gray70', xlab = 'Date', ylab = 'y')
lines(t[-(1:22)],har_fit$fitted.values, col = 'gray40')
lines(t[-(1:J)],h1 - 1.27, col = 'gray30')
lines(t[-(1:J)],h2 - 1.27, col = 'gray20')
lines(t[-(1:J)],h3 - 1.27, col = 'gray10')
par(mar=c(0,0,0,0))
plot(1, type = "n", axes=FALSE, xlab="", ylab="")
legend('top',c('Observed','HAR','1-factor','2-factor', 'ARFIMA'),
       horiz=TRUE,col = c('gray70','gray40','gray30','gray20','gray10'),lty =c(1,1,1,1,1),
       lwd = c(1,1,1,1,1),cex = 0.8, bty = "n")
dev.off()
```